

E3 — MVP: Núcleo Funcional com Primeiras Telas

Solicitação de Entrega | Teoria dos Grafos

Prazo: 28 de abril de 2026 | Peso na nota: 25% da nota final

Contexto

Chegou a hora de sair do papel e entregar software funcionando. O MVP (Minimum Viable Product) é a versão mínima do sistema que executa o algoritmo principal de ponta a ponta — com as primeiras telas reais saindo do código.

Telas reais que funcionam valem mais do que protótipos bonitos: elas provam que a arquitetura está correta, que os dados fluem entre as camadas e que o algoritmo produz algo exibível. Em times ágeis, isso é o que entra na demo do sprint.

O que 'funcionando de ponta a ponta' significa:

O usuário consegue informar um grafo (via arquivo ou entrada manual), o sistema executa o algoritmo e exibe um resultado legível na tela — mesmo que a interface seja simples. Se o algoritmo roda mas nada é exibido, não é MVP.

O que entregar

1. Repositório GitHub

- Repositório público com nome descritivo (ex.: graph-optimizer-dijkstra, grafos-kruskal-logistica)
- Arquivo .gitignore configurado para a linguagem escolhida
- README.md inicial com: nome do projeto, descrição em 2 linhas, como executar o MVP
- Mínimo de 5 commits até esta entrega, com prefixos semânticos: feat:, fix:, test:, docs:, refactor:

2. Estrutura de Diretórios Implementada

A estrutura de diretórios proposta no E2 deve estar criada no repositório:

- src/core/ — classe/módulo do grafo com operações básicas
- src/algorithms/ — ao menos 1 algoritmo implementado em arquivo próprio
- src/io/ — leitura de grafo a partir de arquivo
- tests/ — arquivos de teste
- data/ — ao menos 1 arquivo de grafo de exemplo

3. Implementação do Núcleo

- Classe/módulo do grafo com operações: adicionar vértice, adicionar aresta, consultar adjacência
- Ao menos 1 algoritmo de otimização implementado e executável
- Leitura de grafo a partir de arquivo no formato definido no E2

4. Primeiras Telas Funcionando

A interface não precisa ser polida — precisa funcionar. As telas mínimas esperadas:

- Tela / saída de entrada: onde o usuário informa o arquivo de grafo ou os dados de entrada
- Tela / saída de resultado: onde o algoritmo exibe o resultado de forma legível

Interface pode ser CLI (linha de comando), interface gráfica simples ou web — conforme definido no E2.

5. Testes Unitários

Para cada algoritmo implementado, 3 testes unitários:

- Caso base: entrada válida com resultado esperado conhecido
- Caso com grafo vazio: comportamento com entrada vazia ou nula
- Caso com grafo completo: todos os vértices conectados entre si

Critérios de avaliação

Critério	Peso	Mínimo para aprovação
Repositório público com estrutura correta e .gitignore	15%	Repositório acessível e organizado
Algoritmo executa sem erros e produz resultado correto	30%	Resultado correto em ≥ 2 casos de teste
Telas de entrada e resultado visíveis e operacionais	25%	Fluxo entrada $\rightarrow$ resultado demonstrável
3 testes unitários por algoritmo passando	20%	Ao menos caso base passando
≥ 5 commits semânticos no histórico	10%	Histórico com mais de 1 commit

Dúvidas frequentes

A interface precisa estar bonita?

Não nesta entrega. O MVP prioriza funcionamento. Uma saída de terminal bem organizada já atende. Você vai polir a interface no E4.

Posso entregar mais de um algoritmo?

Sim. Mas garanta que o principal esteja completo e testado antes de adicionar outros. Algoritmo pela metade não conta.

E se os testes não estiverem todos passando?

Entregue o que tiver funcionando. Testes falhando com mensagem de erro clara valem mais do que testes ausentes. Documente no README o que está passando e o que ainda está em desenvolvimento.

Qual o formato esperado do link de entrega?

O link do repositório GitHub público. Garanta que a professora consiga clonar e executar seguindo o README.

Prazo: 10 de maio de 2026 | Valor: 25% da nota final

Dúvidas no fórum da disciplina — não por e-mail particular.